

Name: Eman Fatima
Reg No: BSSE-23S-0025
Program: BSSE
Date: August 25th, 2026

WEEK NO. 1-

PART A: C# AND .NET BASICS

Exercises:-

1. Calculator

Code:

```
Program.cs X
PartA-CSharp > Exercise-01-Calculator > Program.cs
1  Console.Write("Enter first number: ");
2  double number1 = Convert.ToDouble(Console.ReadLine());
3
4  Console.Write("Enter second number: ");
5  double number2 = Convert.ToDouble(Console.ReadLine());
6
7  Console.WriteLine("\n--- Results ---");
8
9  Console.WriteLine("Sum: " + (number1 + number2));
10 Console.WriteLine("Difference: " + (number1 - number2));
11 Console.WriteLine("Product: " + (number1 * number2));
12
13 if (number2 != 0)
14 {
15     Console.WriteLine("Quotient: " + (number1 / number2));
16 }
17 else
18 {
19     Console.WriteLine("Quotient: Cannot divide by zero.");
20 }
```

Output:

```
PS C:\Users\Eman\OneDrive\Desktop\Internship\Nexa Solutions\Week 1\PartA-CSharp\Exercise-01-Calculator> dotnet run
Enter first number: 6
Enter second number: 4

--- Results ---
Sum: 10
Difference: 2
Product: 24
Quotient: 1.5
PS C:\Users\Eman\OneDrive\Desktop\Internship\Nexa Solutions\Week 1\PartA-CSharp\Exercise-01-Calculator> 
```

2. Even or Odd

Code:

```
Program.cs X
PartA-CSharp > Exercise-02-EvenOdd > C# Program.cs
1 Console.WriteLine("Enter a number: ");
2 int number = Convert.ToInt32(Console.ReadLine());
3
4 if (number % 2 == 0)
5 {
6     Console.WriteLine("The number is even.");
7 }
8 else
9 {
10    Console.WriteLine("The number is odd.");
11 }
```

Output:

```
PS C:\Users\Eman\OneDrive\Desktop\Internship\Nexa Solutions\Week 1\PartA-CSharp\Exercise-02-EvenOdd> dotnet run
Enter a number: 6
The number is even.
PS C:\Users\Eman\OneDrive\Desktop\Internship\Nexa Solutions\Week 1\PartA-CSharp\Exercise-02-EvenOdd> dotnet run
Enter a number: 5
The number is odd.
PS C:\Users\Eman\OneDrive\Desktop\Internship\Nexa Solutions\Week 1\PartA-CSharp\Exercise-02-EvenOdd> |
```

3. FizzBuzz

Code:

```
Program.cs X
PartA-CSharp > Exercise-03-FizzBuzz > C# Program.cs
1 for (int i = 1; i <= 100; i++)
2 {
3     if (i % 3 == 0 && i % 5 == 0)
4     {
5         Console.WriteLine("FizzBuzz");
6     }
7     else if (i % 3 == 0)
8     {
9         Console.WriteLine("Fizz");
10    }
11    else if (i % 5 == 0)
12    {
13        Console.WriteLine("Buzz");
14    }
15    else
16    {
17        Console.WriteLine(i);
18    }
19 }
```

Output:

```
PS C:\Users\Eman\OneDrive\Desktop\Internship\Nexa Solutions\Week 1\PartA-CSharp\Exercise-03-FizzBuzz> dotnet run
1
2
Fizz
4
Buzz
Fizz
7
8
Fizz
Buzz
11
Fizz
13
14
FizzBuzz
16
17
Fizz
19
Buzz
Fizz
22
23
Fizz
Buzz
26
Fizz
28
29
FizzBuzz
31
32
Fizz
34
Buzz
Fizz
37
38
Fizz
Buzz
```

```
PS C:\Users\Eman\OneDrive\Desktop\Internship\Nexa Solutions\Week 1\PartA-CSharp\Exercise-03-FizzBuzz> dotnet run
41
Fizz
43
44
FizzBuzz
46
47
Fizz
49
Buzz
Fizz
52
53
Fizz
Buzz
56
Fizz
58
59
FizzBuzz
61
62
Fizz
64
Buzz
Fizz
67
68
Fizz
Buzz
71
Fizz
73
74
FizzBuzz
76
77
Fizz
79
Buzz
```

```
PS C:\Users\Eman\OneDrive\Desktop\Internship\Nexa Solutions\Week 1\PartA-CSharp\Exercise-03-FizzBuzz> dotnet run
Fizz
82
83
Fizz
Buzz
86
Fizz
88
89
FizzBuzz
91
92
Fizz
94
Buzz
Fizz
97
98
Fizz
Buzz
❖ PS C:\Users\Eman\OneDrive\Desktop\Internship\Nexa Solutions\Week 1\PartA-CSharp\Exercise-03-FizzBuzz> █
```

4. Method That Returns a Grade

Code:

```
C# Program.cs X
PartA-CSharp > Exercise-04-Grade > C# Program.cs
1  static string GetGrade(int marks)
2  {
3      if (marks >= 80)
4      {
5          return "A";
6      }
7      else if (marks >= 70)
8      {
9          return "B";
10     }
11     else if (marks >= 60)
12     {
13         return "C";
14     }
15     else if (marks >= 50)
16     {
17         return "D";
18     }
19     else
20     {
21         return "F";
22     }
23 }
24
25 Console.Write("Enter your marks: ");
26 int marks = Convert.ToInt32(Console.ReadLine());
27
28 string grade = GetGrade(marks);
29
30 Console.WriteLine("Grade: " + grade);
```

Output:

```
• PS C:\Users\Eman\OneDrive\Desktop\Internship\Nexa Solutions\Week 1\PartA-CSharp\Exercise-04-Grade> dotnet run
Enter your marks: 89
Grade: A
• PS C:\Users\Eman\OneDrive\Desktop\Internship\Nexa Solutions\Week 1\PartA-CSharp\Exercise-04-Grade> dotnet run
Enter your marks: 75
Grade: B
• PS C:\Users\Eman\OneDrive\Desktop\Internship\Nexa Solutions\Week 1\PartA-CSharp\Exercise-04-Grade> dotnet run
Enter your marks: 62
Grade: C
• PS C:\Users\Eman\OneDrive\Desktop\Internship\Nexa Solutions\Week 1\PartA-CSharp\Exercise-04-Grade> dotnet run
Enter your marks: 50
Grade: D
• PS C:\Users\Eman\OneDrive\Desktop\Internship\Nexa Solutions\Week 1\PartA-CSharp\Exercise-04-Grade> dotnet run
Enter your marks: 36
Grade: F
❖ PS C:\Users\Eman\OneDrive\Desktop\Internship\Nexa Solutions\Week 1\PartA-CSharp\Exercise-04-Grade> |
```

5. Largest and Smallest

Code:

```
C# Program.cs X
PartA-CSharp > Exercise-05-LargestSmallest > C# Program.cs
1  int[] numbers =
2  {
3      25, 10, 45, 3, 78,
4      12, 56, 9, 31, 67
5  };
6
7  int largest = numbers[0];
8  int smallest = numbers[0];
9
10 foreach (int number in numbers)
11 {
12     if (number > largest)
13     {
14         largest = number;
15     }
16
17     if (number < smallest)
18     {
19         smallest = number;
20     }
21 }
22
23 Console.WriteLine("Largest number: " + largest);
24 Console.WriteLine("Smallest number: " + smallest);
```

Output:

```
PS C:\Users\Eman\OneDrive\Desktop\Internship\Nexa Solutions\Week 1\PartA-CSharp\Exercise-05-LargestSmallest> dotnet run
Largest number: 78
Smallest number: 3
PS C:\Users\Eman\OneDrive\Desktop\Internship\Nexa Solutions\Week 1\PartA-CSharp\Exercise-05-LargestSmallest> |
```

6. Menu-Driven Console Program

Code:

```
C# Program.cs X
PartA-CSharp > Exercise-06-Menu > C# Program.cs
1  bool running = true;
2
3  while (running)
4  {
5      Console.WriteLine();
6      Console.WriteLine("==== MENU =====");
7      Console.WriteLine("1. Add");
8      Console.WriteLine("2. View");
9      Console.WriteLine("3. Search");
10     Console.WriteLine("4. Delete");
11     Console.WriteLine("5. Exit");
12     Console.Write("Enter your choice: ");
13
14     string choice = Console.ReadLine();
15
16     switch (choice)
17     {
18         case "1":
19             Console.WriteLine("Add selected.");
20             break;
21
22         case "2":
23             Console.WriteLine("View selected.");
24             break;
25
26         case "3":
27             Console.WriteLine("Search selected.");
28             break;
29
30         case "4":
31             Console.WriteLine("Delete selected.");
32             break;
33
34         case "5":
35             running = false;
36             Console.WriteLine("Exiting program...");
37             break;
```


PartA-CSharp > Exercise-06-Menu > C# Program.cs

```
38
39         default:
40             Console.WriteLine("Invalid choice.");
41             break;
42     }
43 }
```

Output:

```
PS C:\Users\Eman\OneDrive\Desktop\Internship\Nexa Solutions\Week 1\PartA-CSharp\Exercise-06-Menu> dotnet run
C:\Users\Eman\OneDrive\Desktop\Internship\Nexa Solutions\Week 1\PartA-CSharp\Exercise-06-Menu\Program.cs(14,21):
error CS0168: Possible null value to non-nullable type.
```

```
===== MENU =====
```

```
1. Add
2. View
3. Search
4. Delete
5. Exit
```

```
Enter your choice: 4
Delete selected.
```

```
===== MENU =====
```

```
1. Add
2. View
3. Search
4. Delete
5. Exit
```

```
Enter your choice: 2
View selected.
```

```
===== MENU =====
```

```
1. Add
2. View
3. Search
4. Delete
5. Exit
```

```
Enter your choice: 1
Add selected.
```

```
===== MENU =====
```

```
1. Add
2. View
3. Search
4. Delete
5. Exit
```

```
Enter your choice: 3
Search selected.
```

```
PS C:\Users\Eman\OneDrive\Desktop\Internship\Nexa Solutions\Week 1\PartA-CSharp\Exercise-06-Menu> dotnet run
```

```
===== MENU =====
```

```
1. Add
2. View
3. Search
4. Delete
5. Exit
```

```
Enter your choice: 5
Exiting program...
```

```
PS C:\Users\Eman\OneDrive\Desktop\Internship\Nexa Solutions\Week 1\PartA-CSharp\Exercise-06-Menu>
```

PART B: OBJECT-ORIENTED C#

Exercises:-

1. Create a Person Class

Code:

```
PartB-OOP > C# Person.cs > Person
1 reference
1 public class Person
2 {
3     2 references
    public string Name { get; set; }
4     2 references
    public int Age { get; set; }
5
6     0 references
    public Person(string name, int age)
7     {
8         Name = name;
9         Age = age;
10    }
11
12    0 references
    public virtual void Introduce()
13    {
14        Console.WriteLine($"My name is {Name} and I am {Age} years old.");
15    }
16 }
```

Output:

```
PS C:\Users\Eman\OneDrive\Desktop\Internship\Nexa Solutions\Week 1\PartB-OOP> dotnet run
=== PERSON ===
My name is Ahmed and I am 40 years old.
```

2. Create Student and Teacher Classes Using Inheritance

Code:

PartB-OOP >  Student.cs >  Student >  Student(string, int, int, string)

1 reference

```
1 1 ✓ public class Student : Person, IPrintable
2 {
```

2 references

```
3     public int StudentId { get; set; }
```

4 references

```
4     public string Department { get; set; }
```

5

0 references

```
6 6 ✓ public Student(
7     string name,
8     int age,
9     int studentId,
10 10 ✓ string department)
11     : base(name, age)
12 {
13     StudentId = studentId;
14     Department = department;
15 }
```

16

0 references

```
17 17 ✓ public void Study()
18 {
19     Console.WriteLine($"{Name} is studying {Department}.");
20 }
21
```

0 references

```
22 22 ✓ public override void Introduce()
23 {
24 24 ✓     Console.WriteLine(
25         $"I am {Name}, a student studying {Department}."
26     );
27 }
```

```
28     public void Print()
29     {
30         Console.WriteLine(
31             $"Student ID: {StudentId} | Name: {Name} | Department: {Department}"
32         );
33     }
34 }
```

PartB-OOP > C# Teacher.cs > Teacher

```
1 reference
1  public class Teacher : Person
2  {
3      3 references
4      public string Subject { get; set; }
5
6      0 references
7      public Teacher(
8          string name,
9          int age,
10         string subject)
11         : base(name, age)
12     {
13         Subject = subject;
14
15     0 references
16     public void Teach()
17     {
18         Console.WriteLine($"{Name} is teaching {Subject}.");
19     }
20
21     0 references
22     public override void Introduce()
23     {
24         Console.WriteLine(
25             $"I am {Name}, a teacher who teaches {Subject}."
26         );
27     }
28 }
```

Output:

```
=== STUDENT ===
I am Ali, a student studying Computer Science.
Ali is studying Computer Science.
Student ID: 1001 | Name: Ali | Department: Computer Science

=== TEACHER ===
I am Sara, a teacher who teaches Programming.
Sara is teaching Programming.
```

3. Create and Implement an IPrintable Interface

Code:

```
PartB-OOP > C# IPrintable.cs > IPrintable
0 references
1 public interface IPrintable
2 {
3     0 references
4     void Print();
}
```

```
PartB-OOP > C# Program.cs
1 Person person = new Person("Ahmed", 40);
2
3 Console.WriteLine("=== PERSON ===");
4 person.Introduce();
5
6 Student student = new Student(
7     "Ali",
8     21,
9     1001,
10    "Computer Science"
11 );
12
13 Console.WriteLine("\n=== STUDENT ===");
14 student.Introduce();
15 student.Study();
16 student.Print();
17
18 Teacher teacher = new Teacher(
19     "Sara",
20     35,
21     "Programming"
22 );
23
24 Console.WriteLine("\n=== TEACHER ===");
25 teacher.Introduce();
26 teacher.Teach();
```

Output:

```
PS C:\Users\Eman\OneDrive\Desktop\Internship\Nexa Solutions\Week 1\PartB-OOP> dotnet run
=== PERSON ===
My name is Ahmed and I am 40 years old.

=== STUDENT ===
I am Ali, a student studying Computer Science.
Ali is studying Computer Science.
Student ID: 1001 | Name: Ali | Department: Computer Science

=== TEACHER ===
I am Sara, a teacher who teaches Programming.
Sara is teaching Programming.
❖ PS C:\Users\Eman\OneDrive\Desktop\Internship\Nexa Solutions\Week 1\PartB-OOP> █
```

4. Inheritance vs Composition

Inheritance:

Inheritance makes sense when there is an "is-a" relationship between two classes.

For example;

- A Student is a Person.
- A Teacher is a Person.

Therefore, Student and Teacher can inherit from Person.

Composition:

Composition makes sense when there is a "has-a" relationship.

For example;

- A Student has a Course.
- A Teacher has a Department.

A Student is not a Course, so Course should not be the parent class of Student. Instead, a Student can contain or use a Course object.

Inheritance is useful for "is-a" relationships, while composition is useful for "has-a" relationships. Composition can be better when we want to combine different objects without creating a complicated inheritance structure.

PART C: COLLECTIONS, EXCEPTIONS, AND LINQ

Exercises:-

1. Create a List and Add Five Students

Code:

PartC-Collections-LINQ >  Student.cs >  Student

```
1 reference
1 public class Student
2 {
3     1 reference
    public int Id { get; set; }
4     1 reference
    public string Name { get; set; }
5     1 reference
    public string Department { get; set; }
6     1 reference
    public double Marks { get; set; }
7
8     0 references
    public Student(int id, string name, string department, double marks)
9     {
10         Id = id;
11         Name = name;
12         Department = department;
13         Marks = marks;
14     }
15 }
```

PartC-Collections-LINQ >  Program.cs

```
1 List<Student> students = new List<Student>();
2
3 students.Add(new Student(1001, "Ali", "Computer Science", 85));
4 students.Add(new Student(1002, "Sara", "Software Engineering", 92));
5 students.Add(new Student(1003, "Ahmed", "Computer Science", 75));
6 students.Add(new Student(1004, "Fatima", "Artificial Intelligence", 90));
7 students.Add(new Student(1005, "Usman", "Software Engineering", 68));
8
9
10 Console.WriteLine("=== ALL STUDENTS ===");
11
12 foreach (Student student in students)
13 {
14     Console.WriteLine(
15         $"ID: {student.Id}, Name: {student.Name}, Department: {student.Department}, Marks: {student.Marks}"
16     );
17 }
```

Output:

```
PS C:\Users\Eman\OneDrive\Desktop\Internship\Nexa Solutions\Week 1\PartC-Collections-LINQ> dotnet run
=== ALL STUDENTS ===
ID: 1001, Name: Ali, Department: Computer Science, Marks: 85
ID: 1002, Name: Sara, Department: Software Engineering, Marks: 92
ID: 1003, Name: Ahmed, Department: Computer Science, Marks: 75
ID: 1004, Name: Fatima, Department: Artificial Intelligence, Marks: 90
ID: 1005, Name: Usman, Department: Software Engineering, Marks: 68
```

2. Find Students from a Particular Department Using LINQ

Code:

```

20 Console.WriteLine("\n=== COMPUTER SCIENCE STUDENTS ===");
21
22 var computerScienceStudents = students
23     .Where(student => student.Department == "Computer Science");
24
25 foreach (Student student in computerScienceStudents)
26 {
27     Console.WriteLine(
28         $"{student.Name} - {student.Department} - {student.Marks}"
29     );
30 }

```

Output:

```

PS C:\Users\Eman\OneDrive\Desktop\Internship\Nexa Solutions\Week 1\PartC-Collections-LINQ> dotnet run
=== COMPUTER SCIENCE STUDENTS ===
Ali - Computer Science - 85
Ahmed - Computer Science - 75

```

3. Sort Students by Marks

Code:

```

44 Console.WriteLine("\n=== STUDENTS SORTED BY MARKS ===");
45
46 var sortedStudents = students
47     .OrderBy(student => student.Marks);
48
49 foreach (Student student in sortedStudents)
50 {
51     Console.WriteLine(
52         $"{student.Name} - {student.Marks}"
53     );
54 }

```

Output:

```

PS C:\Users\Eman\OneDrive\Desktop\Internship\Nexa Solutions\Week 1\PartC-Collections-LINQ> dotnet run
=== STUDENTS SORTED BY MARKS ===
Usman - 68
Ahmed - 75
Ali - 85
Fatima - 90
Sara - 92

```

4. Find a Student by ID

Code:


```

57 Console.WriteLine("\nEnter student ID to search: ");
58
59 try
60 {
61     int searchId = Convert.ToInt32(Console.ReadLine());
62
63     Student? foundStudent = students
64         .FirstOrDefault(student => student.Id == searchId);
65
66     if (foundStudent != null)
67     {
68         Console.WriteLine(
69             $"Student found: {foundStudent.Name}, {foundStudent.Department}, Marks: {foundStudent.Marks}"
70         );
71     }
72     else
73     {
74         Console.WriteLine("Student not found.");
75     }
76 }
77 catch (FormatException)
78 {
79     Console.WriteLine("Invalid input. Please enter a valid number.");
80 }

```

Output:

```

PS C:\Users\Eman\OneDrive\Desktop\Internship\Nexa Solutions\Week 1\PartC-Collections-LINQ> dotnet run
Enter student ID to search: 1003
Student found: Ahmed, Computer Science, Marks: 75

```

```

PS C:\Users\Eman\OneDrive\Desktop\Internship\Nexa Solutions\Week 1\PartC-Collections-LINQ> dotnet run
Enter student ID to search: 9999
Student not found.

```

```

Enter student ID to search: abc
Invalid input. Please enter a valid number.

```

5. Handle Invalid Numeric Input

Code:

```

59 try
60 {
61     int searchId = Convert.ToInt32(Console.ReadLine());
62
63     Student? foundStudent = students
64         .FirstOrDefault(student => student.Id == searchId);
65
66     if (foundStudent != null)
67     {
68         Console.WriteLine(
69             $"Student found: {foundStudent.Name}, {foundStudent.Department}, Marks: {foundStudent.Marks}"
70         );
71     }
72     else
73     {
74         Console.WriteLine("Student not found.");
75     }
76 }
77 catch (FormatException)
78 {
79     Console.WriteLine("Invalid input. Please enter a valid number.");
80 }

```

Output:

```
Enter student ID to search: abc
Invalid input. Please enter a valid number.
```

PART D: ANGULAR AND TYPESCRIPT BASICS

Exercises:-

1. Create a Simple Angular App That Displays Your Name and Internship Title

Code:

```
PartD-Angular-Student > src > app > <> app.html > div.container
1  <div class="container">
2
3  <h1>AI Software Development Internship</h1>
4
5  <h2>Student Information</h2>
6
7  <p>
8    <strong>Name:</strong>
9    Eman Fatima Janjua
10 </p>
11
12 <p>
13   <strong>Internship Title:</strong>
14   AI Software Development Internship
15 </p>
```

Output:

AI Software Development Internship

Student Information

Name: Eman Fatima Janjua

Internship Title: AI Software Development Internship

2. Create a Student Interface in TypeScript

Code:

```
PartD-Angular-Student > src > app > TS student.ts > Student
1  export interface Student {
2      id: number;
3      name: string;
4      department: string;
5      marks: number;
6  }
```

Output:

AI Software Development Internship

Student Information

Name: Eman Fatima Janjua

Internship Title: AI Software Development Internship

Student List

ID	Name	Department	Marks	Action
1001	Ali	Computer Science	85	<button>Select</button>
1002	Sara	Software Engineering	92	<button>Select</button>
1003	Ahmed	Computer Science	75	<button>Select</button>
1004	Fatima	Artificial Intelligence	90	<button>Select</button>
1005	Usman	Software Engineering	68	<button>Select</button>

3. Create an Array of Students and Display Them in a Table

Code:

PartD-Angular-Student > src > app > TS app.ts > App

```
9   export class App {
13     students: Student[] = [
14       {
15         id: 1001,
16         name: 'Ali',
17         department: 'Computer Science',
18         marks: 85
19       },
20       {
21         id: 1002,
22         name: 'Sara',
23         department: 'Software Engineering',
24         marks: 92
25       },
26       {
27         id: 1003,
28         name: 'Ahmed',
29         department: 'Computer Science',
30         marks: 75
31       },
32       {
33         id: 1004,
34         name: 'Fatima',
35         department: 'Artificial Intelligence',
36         marks: 90
37       },
38       {
39         id: 1005,
40         name: 'Usman',
41         department: 'Software Engineering',
42         marks: 68
43       }
44     ];
```

```
1  <div class="container">
17  <h2>Student List</h2>
18
19  <table>|
20    <thead>
21      <tr>
22        <th>ID</th>
23        <th>Name</th>
24        <th>Department</th>
25        <th>Marks</th>
26        <th>Action</th>
27      </tr>
28    </thead>
29
30    <tbody>
31      @for (student of students; track student.id) {
32
33        <tr>
34          <td>{{ student.id }}</td>
35          <td>{{ student.name }}</td>
36          <td>{{ student.department }}</td>
37          <td>{{ student.marks }}</td>
38
39          <td>
40            <button (click)="selectStudent(student)">
41              Select
42            </button>
43          </td>
44        </tr>
45      }
46    </tbody>
47  </table>
```

Output:

Student List

ID	Name	Department	Marks	Action
1001	Ali	Computer Science	85	<button>Select</button>
1002	Sara	Software Engineering	92	<button>Select</button>
1003	Ahmed	Computer Science	75	<button>Select</button>
1004	Fatima	Artificial Intelligence	90	<button>Select</button>
1005	Usman	Software Engineering	68	<button>Select</button>

4. Add a Button That Selects a Student and Shows Their Details

Code:

```
PartD-Angular-Student > src > app > TS app.ts > App > students
9  export class App {
46      selectedStudent: Student | null = null;
47
48      selectStudent(student: Student): void {
49          | this.selectedStudent = student;
50      }
51  }
```

```
42      <button (click)="selectStudent(student)">
43          |   Select
44      </button>
```

```
1    <div class="container">
52    </table>
53
54    @if (selectedStudent) {
55
56        <div class="details">
57
58            <h2>Selected Student</h2>
59
60            <p>
61                <strong>ID:</strong>
62                {{ selectedStudent.id }}
63            </p>
64
65            <p>
66                <strong>Name:</strong>
67                {{ selectedStudent.name }}
68            </p>
69
70            <p>
71                <strong>Department:</strong>
72                {{ selectedStudent.department }}
73            </p>
74
75            <p>
76                <strong>Marks:</strong>
77                {{ selectedStudent.marks }}
78            </p>
79
80        </div>
81
82    }
83
84 </div>
```

Output:

Student List

ID	Name	Department	Marks	Action
1001	Ali	Computer Science	85	Select
1002	Sara	Software Engineering	92	Select
1003	Ahmed	Computer Science	75	Select
1004	Fatima	Artificial Intelligence	90	Select
1005	Usman	Software Engineering	68	Select

Selected Student

ID: 1001

Name: Ali

Department: Computer Science

Marks: 85

←

→

↺

🏠

🌐 localhost:4200

☆

👤

Finish update

⋮

Student Information

Name: Eman Fatima Janjua

Internship Title: AI Software Development Internship

Student List

ID	Name	Department	Marks	Action
1001	Ali	Computer Science	85	Select
1002	Sara	Software Engineering	92	Select
1003	Ahmed	Computer Science	75	Select
1004	Fatima	Artificial Intelligence	90	Select
1005	Usman	Software Engineering	68	Select

Selected Student

ID: 1003

Name: Ahmed

Department: Computer Science

Marks: 75